

Percolator Launch Security Audit

Augmentation report and final oracle confirmation

Audited repository: ToXMon/percolator-launch-audit

Repository URL: <https://github.com/ToXMon/percolator-launch-audit>

Commit: 0988412a8adbca83e68605b7eae7ef3d154af5

Date: October 2026

Author: Security Engineering

Classification: Private pre-disclosure - do not circulate

VERDICT

Eight confirmed dispositions and two remaining candidates; the creator-retained oracle authority is **CONFIRMED-KNOWN-VARIANT** at 0.93 confidence, with bounded rather than instant price convergence.

1. Executive summary

This review examined the Percolator launch application at commit 0988412a, with emphasis on trust boundaries that can move devnet resources, register markets, authenticate keeper operations, and determine who may influence oracle state. It extends the prior FYEO `full_audit`, which scanned 25 files at fb50865, by reading the current API and security surface, sampling adjacent code and migrations, and reviewing the cross-repository keeper path read-only.

The prior audit's nine findings were re-verified individually. Six are fully fixed, one is partially fixed, and two remain present. This augmentation adds ten current findings: the triage pass recorded seven confirmed and three candidates; the final oracle confirmation moves F-03 to a confirmed-known-variant disposition, leaving eight confirmed dispositions and two remaining candidates. The current confirmed exposure is primarily devnet resource exhaustion and control-plane weakness, plus the creator-oracle integrity flaw. No confirmed high or critical launcher finding was established in the initial launcher-only pass; F-03 is high severity and confirmed against public program source but is a known variant, not novel.

The final oracle confirmation changes the disposition of the highest-impact question. A creator who retains the asset-level `oracle_authority` can switch an empty admin market into `AUTH_MARK` and set an arbitrary in-range target. Listing does not require delegation and trading is not disabled by authority provenance. However, exposed settlement price converges through the crank in bounded steps; an "instant slam" is refuted. This exact missing deviation bound is already described in the team's unmerged public commit 114cee0d, so the issue is a known variant, not novel, and the published percolatorbounty H1 covers a different Pyth-window attack.

10

current findings

8

confirmed dispositions

2

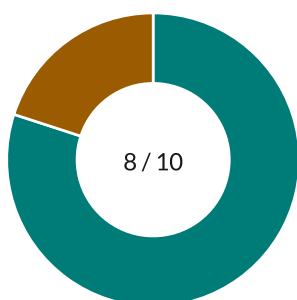
remaining candidates

1

high known variant

Final disposition

Known variant counted as confirmed



READING POSTURE

The final disposition counts F-03 as confirmed-known-variant because its authority and arbitrary target are established in public program source. The two remaining candidates are the proxy-depth spoof path and the public keeper co-sign surface.

All chart marks are SVG vector primitives; no raster screenshots or charts are embedded.

2. Scope, methodology and coverage

The engagement was a static, read-only review. No target-repository changes, public disclosure, exploit execution, devnet write, or mainnet write was performed. The review used the authored report as its spine, the triage JSON as the findings of record, the triage ledger for coverage and regression detail, and the oracle confirmation report for the final F-03 disposition.

Area	Prior FYEO baseline	This augmentation
Repository point	fb50865	0988412a
Files in tree	25 scanned	1,777 in tree
API/security files	Narrow faucet/mint/API subset	24 read deeply; about 80 targeted-sampled; all 46 API routes inventoried by grep
Oracle/keeper	Not the current cross-repo flow	Launcher plus read-only keeper sampling; deployed wrapper/engine source confirmation
Database controls	Not the current broad ledger	RLS grep across all migrations; relevant policies read
Findings	9 baseline	10 current: 8 confirmed dispositions, 2 candidates after final F-03 confirmation
Dynamic/on-chain execution	Not represented	Not performed; final F-03 source confirmation was static public-source analysis

What was read and sampled

- Full or majority reads of 24 security-relevant files: API routes, auth and rate-limit libraries, signer code, market registration, oracle routes and client key-material code.
- Targeted-grep sampling of approximately 80 additional files for secrets, signers, rate limits, fail-open behavior, authorization and value movement; all Supabase migrations were searched for RLS and relevant policies were read.
- Read-only inspection of the external oracle keeper for registration, price reading, circuit breaker and auth-mark push behavior. The final F-03 confirmation additionally inspected the public deployed wrapper and paired engine source.
- Applied the local security checklist for signer and authority validation, vault and withdrawal paths, CPI authority, configuration writes and oracle trust assumptions.

Honest coverage limits

Approximately 1,240 files were not read, including most UI components, hooks, pages, tests, docs and scripts. `useCreateMarket.ts` (more than 3,400 lines) was grep-sampled, `lib/server-rpc.ts` and transaction helpers were skimmed, and SDK internals were trusted. The current final oracle pass did read the paired public program source cited in the confirmation report, but did not build or execute it or verify the deployed binary by transaction.

3. What changed since the prior audit

Six baseline findings are fully fixed, one is partially fixed, and two remain present. The table preserves the strict evidence-backed disposition and current file:line references.

#	Baseline finding	Status	Current evidence
1	RPC proxy origin gate accepted spoofed localhost Origin	FIXED	app/app/api/rpc/route.ts:522-528
2	Faucet/claim gates failed open on DB errors	FIXED	app/lib/faucet-rate-gate.ts:70-91,134; devnet-airdrop/route.ts:130-205
3	devnet-mint-token lacked per-wallet rate gate	FIXED	app/app/api/devnet-mint-token/route.ts:185-199
4	devnet-mint-token amount had no upper cap	STILL PRESENT	app/app/api/devnet-mint-token/route.ts:275-276,388-389 -> F-01
5	devnet-mint-token had no per-wallet rate limit	FIXED	app/app/api/devnet-mint-token/route.ts:185-199
6	Unauthenticated devnet-mirror-mint could drain server SOL	PARTIALLY FIXED	app/app/api/devnet-mirror-mint/route.ts:57,65-76 -> F-02
7	devnet-airdrop claim gate failed open	FIXED	app/app/api/devnet-airdrop/route.ts:130-205
8	Slab secret key in plaintext browser localStorage	STILL PRESENT; accepted trade-off documented	app/lib/inFlightMarket.ts:25,56-57 -> F-10
9	Keeper secret comparison leaked length by early return	FIXED	app/lib/keeper-hmac.ts:87-96,105-110

4. Threat model

The system is a Next.js launcher and API surface around Solana programs, a semi-trusted oracle keeper, Supabase metadata, and server-held devnet signers. The primary security question is not only whether a route authenticates, but which signer or authority can ultimately move value, alter market metadata, or set a price that the on-chain engine consumes.

System description

Public clients discover markets, submit wallet-signed transactions and call devnet funding endpoints. Market creators register metadata and may optionally delegate an asset oracle to the keeper. The keeper reads mainnet DEX pools and pushes marks into delegated devnet markets. On-chain programs enforce trade, liquidation, LP, withdrawal and oracle state rules; app-side display caps and UI controls are not substitutes for those checks.

Trust boundaries

Boundary	Trust and capability	Evidence / control
Public HTTP	Unauthenticated clients can read data, call devnet faucets/mints and submit allowlisted RPC requests, subject to middleware and route limits.	app/middleware.ts; app/app/api/**/route.ts
Authenticated user	Privy access tokens are verified server-side; no broad reviewed server authorization was found.	app/lib/privy-auth.ts
Market creator	Wallet signature proves metadata control, not keeper delegation.	app/app/api/markets/route.ts:1259+; playground/keeper-register
Keeper	Semi-trusted service registers metadata, patches fields and pushes delegated marks.	app/lib/keeper-hmac.ts; external auth-mark-pusher.ts
Admin	Shared admin secrets protect price-cap and bypass paths; Privy admin session is fail-closed.	app/lib/admin-secret.ts; admin-session.ts
Server signers	Mint authority, crank, keeper and SOL faucet can mint, pay fees or sign delegated transactions.	app/lib/devnet-signer.ts; playground-keeper-signer.ts
On-chain program	Signatures, PDAs, authority checks, trades, LP, liquidations and withdrawals are program-controlled.	app/lib/program-ids.ts; paired public source for F-03
Browser key material	Same-origin scripts can read unfinished slab keys needed for reclaim; wallet private keys were not found in app storage.	app/lib/inFlightMarket.ts:25,56-57

Protected assets

- User and LP funds, margin, insurance and settlement integrity on the Solana programs.
- Server-held devnet SOL, mint authority balances and test-token supply.
- Oracle authority, mark targets, effective prices, market registration and keeper credentials.
- Metadata, creator identity proofs, Supabase records and unfinished slab rent/recovery state.

Attack surface and attacker profiles

Attackers include anonymous internet clients seeking devnet resource exhaustion; a malicious or careless market creator retaining oracle authority; a log reader who captures keeper HMAC headers; an XSS attacker reading in-flight slab keys; and a deployment operator who misconfigures Upstash or proxy depth. The external keeper is semi-trusted rather than fully trusted: it can push delegated marks but is constrained by its own circuit breaker and the on-chain authority model. No assumption was made that UI removal prevents direct Solana instruction submission.

5. Findings

Findings are ordered by attacker value and severity. CONFIRMED means the launcher or public-source path is established; CONFIRMED-KNOWN-VARIANT is established but already described in public program history. CANDIDATE means a material prerequisite remains deployment- or binary-dependent. Every finding carries its evidence, path, impact, confidence, status reasoning and remediation inline.

F-03 - Creator-controlled mark price on creator-run admin-oracle markets listed to all users (oracle rug)

CONFIRMED-KNOWN-VARIANT**HIGH**

HIGH - potential user-fund integrity impact if prerequisites hold

CWE: CWE-345

Location: app/app/api/mobile/create-market/route.ts:206-224; related listing app/app/api/markets/route.ts:1259+

Category: Oracle / trust-boundary

CODE EVIDENCE

```
if (oracle_mode !== "admin") { return NextResponse.json({ error: "Only admin is currently supported" }, { status: 400 }); } // ... priceE6 = BigInt(initial_price_e6);
```

Description

The final confirmation read the deployed wrapper source at dcccrypto/percolator-prog 592286b4 and paired engine source at dcccrypto/percolator 35ddd692. A creator-retained asset oracle_authority can switch an empty admin market into AUTH_MARK and submit PushAuthMark tag 63 with any target from 1 through 1,000,000,000,000 E6 (\$0.000001 through \$1,000,000). The handler checks signer, generation, lifecycle, mode, slot and observation sequence, but has no comparison to the prior target or effective price. Listing does not require keeper delegation and trading is not disabled by authority provenance. The effective settlement price is not directly replaced in one push: exposed assets converge toward the target through the crank under max_price_move_bps_per_slot and bounded elapsed slots. This is therefore an arbitrary-target, bounded-convergence oracle rug, not an instant slam.

Attack path

Flow: Plain admin creation -> creator remains asset-level oracle_authority -> creator configures AUTH_MARK before exposure -> market completes and lists without UpdateAssetAuthority(Oracle -> keeper) -> creator signs PushAuthMark with a current slot and fresh observation sequence -> permissionless crank moves effective price toward the arbitrary target -> a liquidation-sized move can harm counterparties after one or a few bounded segments.

1. Create a plain admin market and do not use keeper mode.
2. Complete/list it while retaining the creator oracle key.
3. Before positions exist, configure AUTH_MARK.
4. Attract counterparties.

5. Push a much lower target against longs or a much higher target against shorts.
6. Crank toward the target and observe liquidation or adverse settlement as thresholds are crossed. This was confirmed by static public-source analysis only; no transaction was sent.

IMPACT

High integrity impact and conditional financial loss for counterparties or LPs. Existing exposure activates bounded convergence, so the impact is not an instantaneous arbitrary effective-price write.

LIKELIHOOD

Medium, where a creator can retain authority and attract counterparties.

CONFIDENCE

0.93 (high)

EXPLOITABILITY

Moderate: requires creator-controlled market, counterparties, direct program tooling, and one or more crank segments.

FINAL CONFIRMATION

CONFIRMED-KNOWN-VARIANT - The creator authority, arbitrary in-range target, missing deviation bound, listing path, and absence of an authority-provenance trading block are confirmed in public source. The bounded crank behavior refutes an instant slam. The exact issue is already described by unmerged public commit 114cee0d in the team's own percolator-prog repository, so this is a known variant, not a novel finding. The published percolatorbounty H1 is a different Pyth-window attack.

Verdict confidence: 0.93. Attack value: Counterparty or LP value on a creator-run market; mainnet-relevant class despite current devnet scope.. Weaponizable: true.

Recommended remediation

Merge and deploy an on-chain push-level deviation bound, reusing max_price_move_bps_per_slot and elapsed authenticated slots. Separately require an allowlisted or keeper oracle_authority before a market is shown as publicly tradeable, or make creator-priced markets visibly and technically non-counterparty markets. Add an integration test for an admin market registered without delegation, including attempted mark push and listing.

Precision: the final verdict is CONFIRMED-KNOWN-VARIANT at 0.93 confidence. The creator-retained authority and arbitrary target are confirmed; "instant slam" is refuted because exposed effective price converges through bounded crank steps. The public unmerged commit 114cee0d already describes the missing deviation check, while published percolatorbounty H1 is a different attack and must not be conflated with this finding.

F-01 - devnet-mint-token airdrop amount uncapped (baseline #4 regression)

CONFIRMED

MEDIUM

MEDIUM - devnet resource/value exhaustion

CWE: CWE-190 Location: app/app/api/devnet-mint-token/route.ts:275 Category: Resource exhaustion / value inflation

CODE EVIDENCE

```
const tokensFloat = AIRDROP_USD_VALUE / tokenInfo.priceUsd; const airdropAmount =  
BigInt(Math.floor(tokensFloat * 10 ** decimals));
```

Description

The \$500-USD airdrop converts price to a raw token amount with no upper clamp. priceUsd comes from DexScreener and decimals from token metadata, so a dust-priced token yields a mint of ~5e14 raw tokens per call, drawn from the shared DEVNET_MINT_AUTHORITY_KEYPAIR. The sibling route devnet-airdrop clamps with MIN_RAW/MAX_RAW (route.ts:100); this route has no clamp on either the existing-mint path (:275) or the new-mint path (:388).

Attack path

Flow: POST /api/devnet-mint-token -> per-IP checkMintRateLimit -> tryFaucetGate (per wallet+token 24h) -> fetchTokenInfo(DexScreener) -> mintTo from shared mint authority. The gate is keyed per (wallet, mainnetCA): fresh wallets or fresh CAs repeat the mint. The per-IP limiter (10/min, Upstash, in-memory fallback) is the only cross-wallet bound.

Pick a mainnet token with a tiny priceUsd and valid DexScreener metadata; call the endpoint once per wallet/token pair; receive ~5e14 raw tokens per call. Devnet value only, but inflates mirror mints and drains the shared signer.

IMPACT

low

LIKELIHOOD

high (devnet)

CONFIDENCE

high

EXPLOITABILITY

trivial (single HTTP call)

DISPOSITION

CONFIRMED - Traced end-to-end in code; the only external dependency is the DexScreener price feed, which is the attacker-chosen input. Baseline #4 is 919 commits old and still present verbatim in both airdrop paths.

Verdict confidence: high. Attack value: devnet faucet drain (test tokens, shared signer). Weaponizable: true.

Recommended remediation

Apply the same bounded raw amount policy as devnet-airdrop to both calculation sites. Reject non-finite, non-positive, and out-of-range values before building the transaction. Add a unit test for a dust price and high decimals.

F-02 - Unauthenticated devnet-mirror-mint spends shared wallet SOL (baseline #6, partially fixed)

CONFIRMED

MEDIUM

MEDIUM - devnet resource/value exhaustion

CWE: CWE-306 Location: app/app/api/devnet-mirror-mint/route.ts:57 Category: Missing authentication

CODE EVIDENCE

```
const MINT_RATE_LIMIT_MAX = 10; // ... Upstash slidingWindow limiter, with in-memory fallback when Upstash unconfigured
```

Description

The endpoint remains unauthenticated (by design for the wizard), and every accepted call creates an on-chain SPL mint whose rent and fees are paid by the shared DEVNET_MINT_AUTHORITY wallet. The Upstash 10/min/IP limiter added since the baseline bounds the drain, but the in-memory fallback is per-instance and getClientIp has a misconfig-dependent spoof path (F-06).

Attack path

Flow: POST /api/devnet-mirror-mint -> getClientIp -> Upstash limiter (or in-memory fallback) -> devnet_mints lookup -> create mint via sealed mint-authority signer. No auth of any kind; the only gates are rate limits.

Anonymous caller at >10 req/min/IP (or across instances when Upstash is unset, or with spoofed x-real-ip per F-06) creates mints until the shared wallet's SOL is exhausted.

IMPACT

low

LIKELIHOOD

medium

CONFIDENCE

high

EXPLOITABILITY

easy (scripted HTTP)

DISPOSITION

CONFIRMED - Auth absence and the SOL cost are confirmed in code; the drain rate depends on deployment config (Upstash, TRUSTED_PROXY_DEPTH). Baseline #6 assessed PARTIALLY FIXED.

Verdict confidence: high. Attack value: devnet SOL drain (shared wallet). Weaponizable: true.

Recommended remediation

Prefer a short-lived signed wizard capability bound to the market or creator and one mint operation. At minimum, retain a global shared limiter, enforce a signer balance budget, and fail closed when the global limiter is unavailable.

F-07 - devnet-pre-fund has no per-IP fund limiter

CONFIRMED

MEDIUM

MEDIUM - devnet resource/value exhaustion

CWE: CWE-770 Location: app/lib/prefund-requirement.ts:71 Category: Resource exhaustion

CODE EVIDENCE

NOTE this route has no per-IP fund limiter, unlike /api/playground/faucet, // /api/auto-fund and /api/devnet-airdrop (see lib/fund-ip-rate-limit.ts).

Description

The pre-fund route is documented (in its own shared module) as lacking the per-IP fund limiter every sibling faucet endpoint applies. Only middleware's general 120 req/min/IP and the per-wallet 24h gate apply, and the per-wallet gate is defeated by fresh keypairs. Per call the mint is bounded by MAX_FUNDABLE_REQUIREMENT (10,000 tokens -> 20,000 minted raw) from the shared mint authority.

Attack path

Flow: POST /api/devnet-pre-fund -> middleware 120/min/IP -> per-wallet 24h claim (Blob-backed) -> mint 2x requirement to caller wallet. Fresh keypairs each get a fresh wallet gate.

Cycle fresh devnet keypairs and call pre-fund up to 120/min/IP; drain the shared sim-USDC mint authority at up to 20,000 tokens per call.

IMPACT

low

LIKELIHOOD

high (devnet)

CONFIDENCE

high

EXPLOITABILITY

trivial

DISPOSITION

CONFIRMED - Route code and the module comment both confirm the missing limiter; drain rate bounded by middleware and the per-call cap.

Verdict confidence: high. Attack value: devnet faucet drain (test tokens). Weaponizable: true.

Recommended remediation

Apply the existing fund-ip-rate-limit to this route, with a global shared store and a budget appropriate to market creation. Keep the per-wallet gate as a second control.

F-05 - Keeper HMAC signatures replayable within 5-minute window (no nonce)

CONFIRMED

LOW

LOW - bounded or conditional impact

CWE: CWE-294 Location: app/lib/keeper-hmac.ts:13 Category: Authentication

CODE EVIDENCE

```
const MAX_SIGNATURE_AGE_MS = 5 * 60_000; // 'NOT fixed here... there is still no nonce, so a captured signature is replayable // against that same endpoint within MAX_SIGNATURE_AGE_MS'
```

Description

The keeper<->app HMAC binds method+path+body+timestamp and is timing-safe, but has no nonce: a captured signature is valid for re-submission of the same body to the same endpoint for 5 minutes. PATCH /api/markets/[slab] additionally accepts the legacy unbound signature form (allowLegacyUnbound=true, route.ts:530) during the rollout transition, which weakens the method/path binding.

Attack path

Flow: An observer or log-reader who captures x-keeper-timestamp + x-keeper-signature + raw body can re-submit the identical request for 5 minutes. Endpoints affected: POST /api/oracle-keeper/register, PATCH /api/markets/[slab], POST /api/playground/keeper-register (internal hop).

Capture a keeper HMAC header pair (e.g. from a misconfigured logging path or an http hop - the code enforces https/loopback for KEEPER_INTERNAL_URL, so in practice only log/IPC capture); replay within 5 minutes.

IMPACT

low

LIKELIHOOD

low

CONFIDENCE

high

EXPLOITABILITY

low

DISPOSITION

CONFIRMED - Documented unfixed in the code (#2476); capture vector requires an additional defect (HTTPS enforced, loopback enforced). Low severity.

Verdict confidence: high. Attack value: replay of a privileged write (metadata/registration). Weaponizable: false.

Recommended remediation

Add a nonce or idempotency key stored in a shared TTL store and reject reuse. Remove allowLegacyUnbound after migration telemetry is quiet.

F-08 - Unauthenticated advance-phase route spends CRANK_KEYPAIR SOL (dead instruction in v17)

CONFIRMED

LOW

LOW - bounded or conditional impact

CWE: CWE-306 Location: app/app/api/oracle/advance-phase/route.ts:126 Category: Missing authentication

CODE EVIDENCE

```
const data = encodeAdvanceOraclePhase(); // ... comment: 'AdvanceOraclePhase does not exist in v17  
(removed; was v12 tag 92) - this ALWAYS throws'
```

Description

POST /api/oracle/advance-phase has no authentication; any caller supplies a slabAddress and the route builds and sends a CRANK_KEYPAIR-signed transaction (crank pays fees), rate-limited 60/min/IP. On current v17 deployments the instruction always throws before submission, making it dead code with a live unauthenticated entry point.

Attack path

Flow: POST -> rate limit (60/min/IP) -> loadCrankKeypair -> getLatestBlockhash -> build tx (fee payer = crank) -> send. If v12 tiers were ever re-enabled, anonymous callers could cycle crank-signed txs at 60/min.

Anonymous POST /api/oracle/advance-phase with arbitrary slab addresses; each successful call spends crank SOL for fees. Currently no-ops because the instruction is removed in v17.

IMPACT

low

LIKELIHOOD

low

CONFIDENCE

high

EXPLOITABILITY

trivial

DISPOSITION

CONFIRMED - Auth absence and server signing confirmed; the SOL spend is currently unreachable (instruction removed), hence low.

Verdict confidence: high. Attack value: crank wallet fee drain (devnet). Weaponizable: true.

Recommended remediation

Delete the dead route. If legacy support is required, gate it with a server capability and enforce a signer budget and program-version allowlist.

F-09 - Per-instance in-memory rate-limit fallbacks bypassable on serverless

CONFIRMED

LOW

LOW - bounded or conditional impact

CWE: CWE-770 Location: app/proxy.ts:88 Category: Resource exhaustion

CODE EVIDENCE

```
// GH#2341: unconfigured Upstash silently degrades to the per-instance in-memory limiter // ...  
every cold start gets its own budget, so an attacker spreading requests across // instances bypasses  
it entirely.
```

Description

When UPSTASH_REDIS_REST_URL/TOKEN are unset (or Upstash init fails), the global 120/min, RPC 600/min, fund-endpoint 10/min and mirror-mint 10/min limits all fall back to per-lambda-instance in-memory windows. Production logs the error but does not fail closed.

Attack path

Flow: Client sends requests that land on different serverless instances (normal under load); each instance maintains an independent counter, so effective limits multiply by instance count.

Deploy without Upstash env vars (or wait for a Redis outage), then distribute requests; all per-IP limits collapse to per-instance budgets.

IMPACT

medium

LIKELIHOOD

medium (config-dependent)

CONFIDENCE

high

EXPLOITABILITY

easy given config gap

DISPOSITION

CONFIRMED - Fallback behavior confirmed in code (proxy.ts, upstash-rate-limit.ts, mirror-mint route); exploitation requires a deployment config gap.

Verdict confidence: high. Attack value: rate-limit bypass enabling faucet drains. Weaponizable: true.

Recommended remediation

Treat missing or unhealthy shared rate limiting as fail closed for server-funded routes, or use a platform-enforced global limiter. Add startup and deployment checks that reject production without Upstash configuration.

F-10 - Slab secret key persisted in plaintext in browser localStorage (baseline #8, intentional)

CONFIRMED

LOW

LOW - bounded or conditional impact

CWE: CWE-522 Location: app/lib/inFlightMarket.ts:56 Category: Sensitive data exposure

CODE EVIDENCE

```
slabSecretKey: number[]; // ... window.localStorage.setItem(k, JSON.stringify(state));
```

Description

The in-flight-market recovery flow stores the slab keypair's 64-byte secret key in localStorage so the in-UI ReclaimSlabRent path works after tab close. The header comment (2026-05-12) documents the accepted trade-off: bounded amounts, SDK pin + CSP, and the secret is required for rent reclaim of a half-created slab. An XSS with same-origin read can recover the keypair of an unfinished slab.

Attack path

Flow: saveInFlightMarket() writes {slabAddress, slabSecretKey, ...} to localStorage under 'percolator:in-flight-market:<slab>'. loadLastInFlightMarket()/loadAllInFlightMarkets() read it back for resume/reclaim. No wallet private keys are stored by the app anywhere else.

XSS on the app reads the slab secret; attacker can sign ReclaimSlabRent for the unfinished slab (rent value only) or complete the market creation flow as the slab key.

IMPACT

low

LIKELIHOOD

low

CONFIDENCE

high

EXPLOITABILITY

requires XSS first

DISPOSITION

CONFIRMED - Confirmed present at HEAD; documented intentional. Value bounded to slab rent / an unfinished market's creation path.

Verdict confidence: high. Attack value: slab rent / unfinished-market takeover. Weaponizable: false.

Recommended remediation

Retain localStorage only if recovery UX requires it; minimize lifetime, clear after completion, bind reclaim authorization to the user wallet where possible, and keep CSP and dependency controls. Do not store wallet private keys.

F-06 - getClientIp fails open to spoofable x-real-ip on TRUSTED_PROXY_DEPTH misconfiguration

CANDIDATE

LOW

LOW - bounded or conditional impact

CWE: CWE-290 Location: app/lib/get-client-ip.ts:18 Category: Authentication / spoofing

CODE EVIDENCE

```
const PROXY_DEPTH = Math.max(0, Number(process.env.TRUSTED_PROXY_DEPTH ?? 1)); // ... if (PROXY_DEPTH > 0) { /* XFF peel */ } const realIp = req.headers.get("x-real-ip");
```

Description

A non-numeric TRUSTED_PROXY_DEPTH yields NaN ($\text{Math.max}(0, \text{NaN}) = \text{NaN}$; $\text{NaN} > 0$ is false), silently skipping the XFF chain and keying every per-IP rate limit on the attacker-controllable x-real-ip header. An explicit TRUSTED_PROXY_DEPTH=0 does the same. On Vercel, x-real-ip is platform-set, so exposure is misconfiguration or non-Vercel deploys.

Attack path

Flow: getClientIp() is the key extractor for: proxy.ts global/RPC limits, fund-ip-rate-limit (shared faucet limiter), devnet-mirror-mint, markets/challenge, advance-phase, create-market rate limits.

Set TRUSTED_PROXY_DEPTH to a non-numeric value (or 0) on a deployment, then send x-real-ip with a fresh value per request to bypass every per-IP limit.

IMPACT

medium

LIKELIHOOD

low (config-dependent)

CONFIDENCE

medium

EXPLOITABILITY

easy given config gap

DISPOSITION

CANDIDATE - NaN fail-open path confirmed by code reading; exploitation requires a misconfigured or non-Vercel deployment (defaults are safe).

Verdict confidence: medium. Attack value: rate-limit bypass enabling faucet drains. Weaponizable: true.

Recommended remediation

Require Number.isInteger and an allowed range for TRUSTED_PROXY_DEPTH; fail deployment or startup on invalid values. Ignore client-supplied x-real-ip unless the platform contract guarantees it is overwritten, or derive identity from a trusted edge header.

F-04 - Unauthenticated keeper co-sign endpoint exposes privileged partial signatures

CANDIDATE

LOW

LOW - bounded or conditional impact

CWE: CWE-306 Location: app/app/api/playground/keeper-cosign/route.ts:267 Category: Missing authentication

CODE EVIDENCE

```
keeper.partialSign(tx); // ... 'No auth header needed - the keeper co-sign is safe to expose publicly'
```

Description

POST /api/playground/keeper-cosign requires no authentication: any caller can obtain a keeper-keypair partial signature over ConfigureAuthMark + UpdateAssetAuthority(oracle -> keeper) for any (slab, assetIndex<=13), with an attacker-chosen initialMarkE6 <= \$1M. Safe by construction for foreign markets (the current oracle authority must co-sign; deployer is fee payer; deployer===keeper aliasing rejected), so the exposure is the privileged-signature-for-the-internet surface itself.

Attack path

Flow: POST -> validate pubkeys/price (MAX_PRICE_E6 clamp at :153) -> read live marketId/control seqs -> build ConfigureAuthMark + UpdateAssetAuthority -> keeper.partialSign -> return partial tx for the creator to co-sign.

Anonymous caller requests co-signs for arbitrary (slab, assetIndex) pairs; only their own markets complete.

Residual risk: v18 CAS/signer-aliasing edge cases across the 14 asset slots, and zero-cost keeper-keypair signature minting for griefing.

IMPACT

low

LIKELIHOOD

low

CONFIDENCE

medium

EXPLOITABILITY

trivial

DISPOSITION

CANDIDATE - Route is unauthenticated by documented design and safe under the traced signer model; the residual risk (v18 multi-asset CAS) needs the deep pass to rule out.

Verdict confidence: medium. Attack value: oracle delegation on self-owned markets. Weaponizable: false.

Recommended remediation

Keep the public design only if on-chain tests prove the two-party boundary for every asset index and v18 sequence/epoch combination. Otherwise bind requests to a creator proof and add per-creator and endpoint abuse limits.

6. Prioritised remediation roadmap

Order reflects attacker value first, then engineering effort. Quick wins close concrete server-funded exposure; structural work addresses authority provenance and residual trust assumptions.

Quick wins

- F-01: cap devnet-mint-token in both calculation sites and reject invalid arithmetic.
- F-07: apply the shared fund IP limiter to devnet-pre-fund while retaining its wallet gate.
- F-08: delete or hard-gate the dead advance-phase route and its server-signer path.
- F-09: fail closed when shared Upstash limiting is absent or unhealthy; add startup checks.
- F-06: reject invalid proxy-depth configuration and document the trusted edge header contract.
- F-05: add shared TTL nonce/idempotency protection, then remove legacy-unbound HMAC acceptance.

Structural work

- F-03: before broad market launch, require keeper delegation before listing/trading or clearly and technically isolate creator-priced markets. Merge and deploy the on-chain deviation bound; do not rely on display sanitization.
- F-04: test exact v18 binary behavior for every asset slot, stale sequence, wrong authority and signer aliasing; bind public requests to creator proof if needed.
- F-10: decide explicitly whether bounded localStorage recovery remains acceptable; if it does, minimize lifetime and expiry.
- Regression: retain tests for production localhost Origin rejection, DB-gate fail-closed behavior, wallet claim reservation and fixed-length HMAC comparison.

7. Limitations and residual risk

- Static review cannot prove every deployed Solana binary matches SDK or source assumptions. F-03 was confirmed against cited public source, but no transaction or deployed-binary verification was performed.
- No dynamic request, transaction, exploit, devnet write, mainnet write or drain loop was run. Attack paths are reproducible review procedures, not claims of execution.
- The external keeper was sampled read-only; its deployed configuration, registry contents, authority ownership and operational security remain outside this review.
- Most UI, hooks, pages, tests, docs and scripts were not read. SDK internals and some transaction helpers were trusted.
- Devnet faucet findings affect test assets and server-held devnet SOL, not confirmed mainnet user funds. F-03 is the only current finding with a plausible direct user-fund impact.
- The known upstream Pyth-window manipulation disclosure is intentionally excluded as already known.

8. Appendix

Coverage ledger

Deep read: 24 files spanning RPC, market registration, market PATCH, challenge, price-cap, keeper register, advance-phase, oracle resolve, keeper co-sign, faucet/mint routes, auth libraries, rate gates, client key material, Supabase and selected external keeper files.

Targeted sample: approximately 80 files for secrets, localStorage, signers, authorization, fail-open behavior, rate limits and environment variables.

Not read: approximately 1,240 files, including most UI components, hooks, pages, e2e/tests, docs and scripts. The full market-creation wizard and SDK internals were not traced.

Methodology notes

Evidence commands included repository commit verification, targeted source reads at every cited finding location, migration/RLS grep, secret/signer/rate-limit inventory and read-only external keeper inspection. Findings distinguish measured facts from inference and unverified deployment behavior. No exploit code was executed and no target repository or public disclosure artifact was modified.

Provenance and source integrity

The prior FYEO report was produced by an AI audit scanner. This work extends that report rather than replacing it: all nine prior findings were re-verified individually against commit 0988412a. The machine-readable source of truth for the ten current findings is `/Users/tolushekoni/agent-workspace/data/percolator-audit-triage/triage-findings.json`; its recorded SHA-256 is `252998ce08ee85a800a1f043a573f269b3dbba0eafed0470885d00f61c8125ee`. The final oracle disposition is taken from the separate confirmation report and is reproduced precisely here.

FINAL RECORD

CONFIRMED-KNOWN-VARIANT - 0.93 confidence. Creator-retained oracle authority can set an arbitrary in-range target; listing does not require delegation; trading is not disabled; bounded crank convergence refutes an instant slam; the issue is already known in public unmerged program history and is distinct from published percolatorbounty H1.